

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 13 LECTURE 1

VFS – FILE LOCKING

WISH YOU ALL THE BEST

Virtual File System



- **File Locking**
- **When a file can be accessed by more than one process, a synchronization problem occurs.**
- **What happens if two processes try to write in the same file location? Or again, what happens if a process reads from a file location while another process is writing into it?**
- **In traditional Unix systems, concurrent accesses to the same file location produce unpredictable results.**
- **However, Unix systems provide a mechanism that allows the processes to lock a file region so that concurrent accesses may be easily avoided.**



Virtual File System



➤ File Locking

- The POSIX standard provides a file-locking mechanism based on the **fcntl() system call**.
- It is possible to lock an arbitrary region of a file (even a single byte) or to lock the whole file (including data appended in the future).
- Because a process can choose to lock only a part of a file, it can also hold multiple locks on different parts of the file.
- This kind of lock does not keep out another process that is ignorant of locking.
- Like a semaphore used to protect a critical region in code, the lock is considered "advisory" because it doesn't work unless other processes cooperate in checking the existence of a lock before accessing the file.
- Therefore, **POSIX's locks are known as advisory locks**.



Virtual File System



- **File Locking**
- Traditional BSD variants implement advisory locking through the `flock( )` system call.
- This call does not allow a process to lock a file region, only the whole file.
- Traditional System V variants provide the `lockf( )` library function, which is simply an interface to `fcntl( )`.
- More importantly, System V Release 3 introduced **mandatory locking**: the kernel checks that every invocation of the `open( )`, `read( )`, and `write( )` system calls does not violate a mandatory lock on the file being accessed.
- Therefore, mandatory locks are enforced even between noncooperative processes.



Virtual File System



➤ File Locking

- Oddly enough, a process may still unlink (delete) a file even if some other process owns a mandatory lock on it!
- This perplexing situation is possible because when a process deletes a file hard link, it does not modify its contents, but only the contents of its parent directory.
- Whether processes use advisory or mandatory locks, they can use both shared read locks and exclusive write locks.
- Several processes may have read locks on some file region, but only one process can have a write lock on it at the same time.
- Moreover, it is not possible to get a write lock when another process owns a read lock for the same file region, and vice versa.



Virtual File System

➤ File Locking – fcntl system call

❑ Advisory record locking

- Linux implements traditional ("process-associated") UNIX record locks, as standardized by POSIX.
- The fcntl system call allows a program to place a read lock or a write lock on a file, somewhat analogous to the mutex locks.

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
int fcntl(int fd, int cmd, struct flock *lock);
```

- **fcntl()** performs one of the operations determined by *cmd* on the open file descriptor *fd*.
- **fcntl()** can take an optional third argument. The third argument, *lock*, is a pointer to a structure flock

Virtual File System



➤ File Locking – fcntl system call

- The Second Argument cmd can be F_SETLK, F_SETLKW, and F_GETLK which are used to acquire, release, and test for the existence of record locks (also known as byte-range, file-segment, or file-region locks).
- The third argument, lock, is a pointer to a structure that has at least the following fields (in unspecified order).

```
struct flock {  
    ...  
    short l_type;          /* Type of lock: F_RDLCK, F_WRLCK, F_UNLCK */  
    short l_whence;        /* How to interpret l_start: SEEK_SET, SEEK_CUR, SEEK_END */  
    off_t l_start;          /* Starting offset for lock */  
    off_t l_len;            /* Number of bytes to lock */  
    pid_t l_pid;           /* PID of process blocking our lock(set by F_GETLK and F_OFD_GETLK) */  
    ...  
};
```

- The l_whence, l_start, and l_len fields of this structure specify the range of bytes we wish to lock. Bytes past the end of the file may be locked, but not bytes before the start of the file.
- l_start is the starting offset for the lock, and is interpreted relative to either: the start of the file (if l_whence is SEEK_SET); the current file offset (if l_whence is SEEK_CUR); or the end of the file (if l_whence is SEEK_END). In the final two cases, l_start can be a negative number provided the offset does not lie before the start of the file.
- l_len specifies the number of bytes to be locked. If l_len is positive, then the range to be locked covers bytes l_start up to and including l_start+l_len-1. Specifying 0 for l_len has the special meaning: lock all bytes starting at the location specified by l_whence and l_start through to the end of file, no matter how large the file grows. if l_len is negative, the interval described by lock covers bytes l_start+l_len up to and including l_start-1.

Virtual File System

➤ File Locking – fcntl system call

- A read lock is placed on a readable file descriptor, and a write lock is placed on a writable file descriptor.
- More than one process may hold a read lock on the same file at the same time, but only one process may hold a write lock, and the same file may not be both locked for read and locked for write.
- Note that placing a lock does not actually prevent other processes from opening the file, reading from it, or writing to it, unless they acquire locks with fcntl as well.
- To place a lock on a file, first create and zero out a struct flock variable. Set the l_type field of the structure to F_RDLCK for a read lock or F_WRLCK for a write lock.
- Then call fcntl, passing a file descriptor to the file, the F_SETLCKW operation code, and a pointer to the struct flock variable.
- If another process holds a lock that prevents a new lock from being acquired, fcntl blocks until that lock is released.

Virtual File System

➤ File Locking – fcntl system call

- The **`l_type`** field can be used to place a read (**`F_RDLCK`**) or a write (**`F_WRLCK`**) lock on a file.
- Any number of processes may hold a read lock (shared lock) on a file region, but only one process may hold a write lock (exclusive lock).
- An exclusive lock excludes all other locks, both shared and exclusive. A single process can hold only one type of lock on a file region; if a new lock is applied to an already-locked region, then the existing lock is converted to the new lock type.

`F_SETLK` (struct flock *)

Acquire a lock (when **`l_type`** is **`F_RDLCK`** or **`F_WRLCK`**) or release a lock (when **`l_type`** is **`F_UNLCK`**) on the bytes specified by the **`l_whence`**, **`l_start`**, and **`l_len`** fields of lock. If a conflicting lock is held by another process, this call returns -1 and sets **`errno`** to **`EACCES`** or **`EAGAIN`**.

`F_SETLKW` (struct flock *)

As for **`F_SETLK`**, but if a conflicting lock is held on the file, then wait for that lock to be released. If a signal is caught while waiting, then the call is interrupted and (after the signal handler has returned) returns immediately (with return value -1 and **`errno`** set to **`EINTR`**).

`F_GETLK` (struct flock *)

On input to this call, lock describes a lock we would like to place on the file. If the lock could be placed, **`fcntl()`** does not actually place it, but returns **`F_UNLCK`** in the **`l_type`** field of lock and leaves the other fields of the structure unchanged.

- In order to place a read lock, *fd* must be open for reading. In order to place a write lock, *fd* must be open for writing. To place both types of lock, open a file for both read-write.

Virtual File System

➤ File Locking – use of fcntl and flock

/* program to demonstrate file locking – lock_file.c */

```
#include <fcntl.h>
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
int main (int argc, char* argv[])
```

```
{
```

```
char* file = argv[1];
```

```
int fd;
```

```
struct flock lock;
```

```
printf ("opening %s\n", file);
```

```
/* Open a file descriptor to the file. */
```

```
fd = open (file, O_WRONLY);
```

```
printf ("locking\n");
```

```
/* Initialize the flock structure. */
```

```
memset (&lock, 0, sizeof(lock));
```

```
lock.l_type = F_WRLCK;
```

```
/* Place a write lock on the file. */
```

```
fcntl (fd, F_SETLKW, &lock);
```

```
printf ("locked; hit Enter to unlock... ");
```

```
/* Wait for the user to hit Enter. */
```

```
getchar ();
```

```
printf ("unlocking\n");
```

```
/* Release the lock. */
```

```
lock.l_type = F_UNLCK;
```

```
fcntl (fd, F_SETLKW, &lock);
```

```
close (fd);
```

```
return 0;
```

```
}
```

Compile and run the program on a file in one xterm

```
$gcc -o lock_file lock_file.c
```

```
./lock_file lock_file.c
```

```
opening lock_file.c
```

```
locking
```

```
locked; hit Enter to unlock...
```

Now open another xterm and try running the lock_file on the same file

```
$ ./lock_file lock_file.c
```

```
opening lock_file.c
```

```
locking
```

Note that the second instance is blocked while attempting lock the same file

Go back to the first window and press Enter:

```
unlocking
```

The program running in the second window immediately acquires the lock.

```
locked; hit Enter to unlock...
```

Virtual File System

❑ Linux File Locking - flock() system call

- flock() - apply or remove an advisory lock on an open file

```
#include <sys/file.h>
```

```
int flock(int fd, int operation);
```

- Apply or remove an advisory lock on the open file specified by *fd*. The argument *operation* is one of the following:

➤ LOCK_SH

- Place a shared lock. More than one process may hold a shared lock for a given file at a given time

➤ LOCK_EX

- Place an exclusive lock. Only one process may hold an exclusive lock for a given file at a given time

➤ LOCK_UN

- Remove an existing lock held by this process.

- A call to **flock()** may block if an incompatible lock is held by another process.
- To make a nonblocking request, include **LOCK_NB** (by ORing) with any of the above operations.

Virtual File System



❑ Linux File Locking

- Linux supports all types of file locking: **advisory and mandatory locks, plus the `fcntl( )` and `flock( )` system calls (`lockf( )` is implemented as a standard library function).**
- The expected behavior of the `flock( )` system call in every Unix-like operating system is to produce advisory locks only, without regard for the `MS_MANDLOCK` mount flag.
- In Linux, however, a special kind of `flock( )`'s mandatory lock is used to support some proprietary network filesystems .
- **It is the so-called share-mode mandatory lock; when set, no other process may open a file that would conflict with the access mode of the lock.**
- Use of this feature for native Unix applications is discouraged, because the resulting source code will be nonportable.



❑ Linux File Locking

- Another kind of `fcntl( )`-based mandatory lock called **lease** has been introduced in Linux.
- When a process tries to open a file protected by a lease, it is blocked as usual.
- However, the process that owns the lock receives a signal.
- Once informed, it should first update the file so that its content is consistent, and then release the lock.
- If the owner does not do this in a well-defined time interval (tunable by writing a number of seconds into `/proc/sys/fs/lease-break-time`, usually 45 seconds), the lease is automatically removed by the kernel and the blocked process is allowed to continue.





❑ Linux File Locking

A process can **get or release an advisory file lock on a file** in two possible ways:

- **By issuing the flock() system call.** The two parameters of the system call are the fd file descriptor, and a command to specify the lock operation. The lock applies to the whole file.
- **By using the fcntl() system call.** The three parameters of the system call are the fd file descriptor, a command to specify the lock operation, and a pointer to a flock structure. A couple of fields in this structure allow the process to specify the portion of the file to be locked. Processes can thus hold several locks on different portions of the same file.
- Both the fcntl() and the flock() system call may be used on the same file at the same time, but a file locked through fcntl() does not appear locked to flock(), and vice versa.
- This has been done on purpose in order to avoid the deadlocks occurring when an application using a type of lock relies on a library that uses the other type.



Virtual File System

❑ Linux File Locking – Mandatory Locking

Handling mandatory file locks is a bit more complex. Here are the steps to follow:

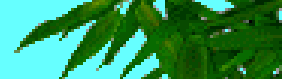
1. Mount the filesystem where mandatory locking is required using the **-o mand** option in the **mount** command, which sets the MS_MANDLOCK flag in the mount() system call. The default is to disable mandatory locking.
2. Mark the files as candidates for mandatory locking by setting their set-group bit (SGID) and clearing the group-execute permission bit. Because the set-group bit makes no sense when the group-execute bit is off, the kernel interprets that combination as a hint to use mandatory locks instead of advisory ones.

Virtual File System

❑ Linux File Locking – Mandatory Locking

- To make use of mandatory locks, **mandatory locking must be enabled both on the filesystem that contains the file to be locked, and on the file itself.**
- **Mandatory locking is enabled on a filesystem using the "-o mand" option to mount command, or the MS_MANDLOCK flag for mount system call.**
- **Mandatory locking is enabled on a file by disabling group execute permission on the file and enabling the set-group-ID permission bit using chmod.**

Virtual File System



❑ Linux File Locking – Mandatory Locking

- The **setuid** (set user id) is a permission bit, that **allows the users to exec a program with the permissions of its owner.**
- The **setgid** (set group id) is a bit that **allows the user to exec a program with the permissions of the group owner.**
- The setuid and setgid can be set with the **chmod** command, like any other permission bits.
- The setuid and setgid can be set in both the octal and the human readable format.
- To add the setuid add the +s bit for the user: **chmod u+s filename**
\$ chmod u+s file1
- To remove the setuid bit use the -s argument with the chmod command: **chmod u-s filename**
\$ chmod u-s file1
- To set the setgid bit on a file, add the +s argument for the group, with **chmod g+s filename:**
\$ chmod g+s file2
- To remove the setgid use -s for the group: **chmod g-s filename**
\$ chmod g-s file2
- To set the setuid in the octal form, place a 4 in front of the three permission bits.
\$ chmod 4777 file1
\$ chmod 4764 file2
- To set the setgid in the octal form, add a 2 before the three permission digits
\$ chmod 2777 myscript
\$ chmod 2764 myscript
- To add both the setuid and setgid place a 6 in front of the three permission bits: **chmod 6777 filename**
- To remove the setuid and setgid place a 0 in front of the three permission bits: **chmod 0777 filename**
- To Find files with both setuid and setgid : **find / -type f -perm /6000**
- To Find only the files with setuid: **find / -type f -perm /4000**
- To Find only the files with setgid: **find / -type f -perm /2000**

Virtual File System



❑ Linux File Locking

- **Handling leases is much simpler than handling mandatory locks:** it is sufficient to invoke a `fcntl( )` system call with a `F_SETLEASE` or `F_GETLEASE` command.
- Another `fcntl( )` invocation with the `F_SETSIG` command may be used to change the type of signal to be sent to the lease process holder.
- Besides the checks in the `read( )` and `write( )` system calls, the kernel takes into consideration the existence of mandatory locks when servicing all system calls that could modify the contents of a file.
- For instance, an `open( )` system call with the `O_TRUNC` flag set fails if any mandatory lock exists for the file.



SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 12 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010

OPERATING SYSTEM

END OF

WEEK 13 LECTURE 1

VIRTUAL FILE SYSTEM

(VFS)-PART2

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG